

```
$ gcc hello.c -o hello
```

What happens between `hello.c` and `./hello`

One command, four stages, five files. Then a first look inside a compiler: frontends, intermediate representations and backends.



All listings in this deck are real output from GCC 13.3 and Clang 21 on x86-64 Linux, trimmed to fit a slide.

gcc is a driver that runs other programs

The gcc command barely compiles anything itself. It works out which tools to run, in which order, and passes files between them.



Ask gcc to show its work

`gcc -v hello.c -o hello`

```
# trimmed: only the three programs gcc starts
.../13/cc1 -quiet ... hello.c ... -o /tmp/ccPQKXkN.s
as --64 -o /tmp/ccYLfJ9K.o /tmp/ccPQKXkN.s
.../13/collect2 ... -o hello ... /tmp/ccYLfJ9K.o ...
```

The in-between files go to `/tmp` and are deleted at the end. The stop flags let you keep them and look inside.

Preprocessing runs inside `cc1`. The classic standalone preprocessor `cpp` also exists. `collect2` is a thin wrapper that calls the real linker, `Ld`.

Our running example

We follow this file through every stage and look at what each program leaves behind.

hello.c 9 lines, 138 bytes

```
#include <stdio.h>

#define GREETING "Hello, world!"

int main(void) {
    // print a greeting
    printf(GREETING "\n");
    return 0;
}
```

It has one of each thing a stage cares about: an include, a macro, a comment, a call into the C library.

Files the build produces ls -l

File	What it is	Size
hello.c	your C source	138 B
hello.i	C with headers pasted in, 822 lines	21 KB
hello.s	x86-64 assembly text	668 B
hello.o	object file: machine code, not runnable	1.5 KB
hello	executable program	16 KB

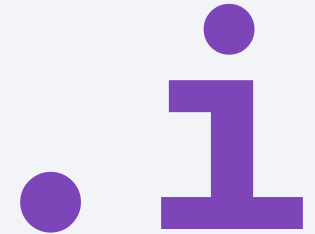
Ask the class: why does the file grow 150 times in the first step, then shrink by 30 times in the next?



Stage 1 of 4

Preprocessing: text in, text out

The preprocessor follows every line that starts with `#`. It does not understand C, it only edits text.



822 lines from 9

Pastes in `#include` files

One line brought in 26 headers, including the declaration of `printf`.

Expands `#define` macros

`GREETING` becomes the string.

Keeps or drops `#if` blocks

Handy for platform-specific code.

Strips comments, adds line markers

Errors still point at `hello.c`.

```
$ gcc -E hello.c -o hello.i
```

`hello.i`

excerpt; `printf` is on line 476

```
# 1 "/usr/include/stdio.h" 1 3 4
...
extern int printf (const char *__restrict __format, ...);
...
# 5 "hello.c"
int main(void) {
    comment removed, empty line kept
    printf("Hello, world!" "\n");
    return 0;
}
```

The preprocessor does not know arithmetic

Macros are pure text substitution. Precedence and types only matter later, once the compiler sees the result.

sq.c

looks fine

```
#define SQUARE(x) x * x

int r = SQUARE(1 + 2);
```

gcc -E -P sq.c

after

```
int r = 1 + 2 * 1 + 2;
```

So r is 5, not 9. Multiplication binds first.

the fix

parenthesise everything

```
#define SQUARE(x) ((x) * (x))
```

Still unsafe with side effects: `SQUARE(i++)` increments twice. In modern C, prefer a `static inline` function.

add.h

include guard

```
#ifndef ADD_H
#define ADD_H
int add(int a, int b);
#endif
```

Same idea, used on purpose: `#ifndef` keeps a header from being pasted twice when several files include it. The first copy defines `ADD_H`, so every later copy is dropped.



Stage 2 of 4

Compilation: C becomes assembly

This is the real compiler, cc1. It checks your C, optimizes it and writes human-readable assembly for the target CPU.

.S

assembly for one CPU

hello.s

x86-64, unwind info removed

```
.section .rodata
.LC0:
.string "Hello, world!"
.text
.globl main
main:
endbr64
pushq %rbp
movq %rsp, %rbp
leaq .LC0(%rip), %rax
movq %rax, %rdi
call puts@PLT
movl $0, %eax
popq %rbp
ret
```

```
$ gcc -S hello.c -o hello.s
```

The string went to read-only data

.rodata, and the code loads its address into a register.

Where did printf go?

GCC knows the C library. A printf of a plain string ending in \n is the same as puts, so it calls that instead.

Arguments travel in registers

The first one in %rdi, the return value in %eax.

How cc1 gets from C to this is Part 2 of the lecture.



Stage 3 of 4

Assembling: text becomes bytes

The assembler `as` turns each instruction into machine code. It works on one file and knows nothing about the others.



object file, not runnable

`objdump -d hello.o`

address, bytes, instruction

```

0000000000000000 <main>:
 0:  f3 0f 1e fa      endbr64
 4:  55               push    %rbp
 5:  48 89 e5         mov     %rsp,%rbp
 8:  48 8d 05 00 00 00 00 lea     0x0(%rip),%rax
 f:  48 89 c7         mov     %rax,%rdi
12:  e8 00 00 00 00    call   17 <main+0x17>
17:  b8 00 00 00 00    mov     $0x0,%eax
1c:  5d              pop     %rbp
1d:  c3              ret
  
```

```

$ gcc -c hello.c -o hello.o
$ file hello.o
ELF 64-bit LSB relocatable, x86-64
  
```

Holes in the code. Where will the string live? Where is `put s`? The assembler can't know, so it writes zeros and leaves a note for the linker.

Addresses in an object file start at 0. The real ones are chosen at link time.

An object file is machine code plus a to-do list

Besides the bytes, `hello.o` records what it defines, what it needs from elsewhere, and which bytes must be patched.

Symbol table

`nm hello.o`

```
000000000000000000 T main
                  U puts
```

- T** defined here, in the code (text) section
- U** undefined: used here, must come from somewhere else

Relocations

`readelf -r hello.o, trimmed`

```
Offset  Type          Sym. Name + Addend
00000b  R_X86_64_PC32      .rodata - 4
000013  R_X86_64_PLT32     puts - 4
```

Offsets `0xb` and `0x13` are exactly the two holes from the previous slide. Each entry tells the linker what address to fill in there.

Common ELF sections

`readelf -S hello.o`

.text	.rodata	.data	.bss	.symtab	.rela.text
machine code	constants, string literals	initialized globals	zero-initialized globals, take no file space	the symbol table above	the relocations above



Stage 4 of 4

Linking: fill the holes, join the pieces

The linker `Ld` combines your object files with startup code and libraries into one program.



Resolve symbols

Find a definition for every `U`.

Merge sections

All `.text` together, all
`.data` together.

Relocate

Choose final addresses, patch
every hole.

Add startup code

Execution begins before
`main`.

What gcc hands to ld

from `gcc -v`

```
Scrt1.o crti.o crtbeginS.o  
hello.o  
-lgcc -lgcc_s -lc -lgcc -lgcc_s  
crtendS.o crtn.o
```

`-lc` is the C library, where `puts` lives. The `crt` files are C runtime startup code.

`nm hello`

after linking

```
00000000000001060 T _start  
00000000000001149 T main  
U puts@GLIBC_2.2.5
```

`main` has a real address now. `puts` is still missing: it comes from `libc.so.6` when the program starts.

Run it

```
$ gcc hello.o -o hello  
$ ./hello  
Hello, world!
```

Start-up order: `_start`, then `__libc_start_main`, then `main`, then `exit` with its return value.

Stage 4 of 4, continued

Static or dynamic linking

Library code can be copied into your program at build time, or found and loaded each time the program starts.

Dynamic (the default)

```
$ gcc hello.o -o hello
$ ldd hello
    linux-vdso.so.1
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
    /lib64/ld-linux-x86-64.so.2
```

The executable only records "I need libc.so.6". At start-up, the dynamic loader `ld-linux` maps the library into memory and connects `puts`.

Small files, one shared copy in memory, library bug fixes apply without rebuilding.

Size of the same hello world

dynamic



static



`ls -l`

15,960 B

785,360 B

Static

```
$ gcc -static hello.o -o hello_static
$ ldd hello_static
    not a dynamic executable
```

The linker copies the needed parts of `libc` into the file itself. Nothing is looked up at start-up.

Runs on machines without matching libraries, but is much bigger and must be relinked to pick up fixes.

Two source files, one program

Each .c file goes through the first three stages alone. Only the linker sees them together.



While compiling `main.c`, the compiler only needs the **declaration** from the header to check the call. The **definition** arrives at link time. Change `add.c`, and only `add.o` needs rebuilding, then a relink. That is exactly what make automates.

Which stage failed? Read the error

Knowing the stages turns confusing error messages into a quick diagnosis.

Preprocessor

```
$ gcc -c e1.c
e1.c:1:10: fatal error:
missing.h:
    No such file or directory
    1 | #include "missing.h"
```

Clue: a file path at an #include

Wrong header name, or add its folder with
-I dir.

Compiler

```
$ gcc -c e2.c
e2.c:1:28: error: expected ',',
      or ';' before 'return'
      1 | int main(void) { int x =
      1 | return x; }
```

Clue: file:line:column

Syntax and type errors in your C. Fix the
code at that spot, or just before it.

Linker

```
$ gcc main2.c -o main2
main2.c:(.text+0x13): undefined
reference to `add'
collect2: error: ld returned 1
exit status
```

Clue: ld, collect2, a section offset

Declared but never defined, or you forgot
a .c/.o file or -l library.

The other classic linker error is the opposite: multiple definition of `add', when two files both define the same function. A common cause is a function body written inside a header.

Cheat sheet

One flag per stage, one tool per file type.

Stage	Program	Stop with	Output	Look inside with
 Preprocess	<code>cc1 -E</code> (or <code>cpp</code>)	<code>gcc -E</code>	<code>.i</code> expanded C text	<code>less</code> , <code>grep</code>
 Compile	<code>cc1</code>	<code>gcc -S</code>	<code>.s</code> assembly text	any text editor
 Assemble	<code>as</code>	<code>gcc -c</code>	<code>.o</code> relocatable ELF object	<code>objdump -d</code> , <code>nm</code> , <code>readelf -r</code>
 Link	<code>ld</code> (via <code>collect2</code>)	<code>gcc</code> with no stop flag	executable ELF	<code>ldd</code> , <code>nm</code> , <code>readelf -h</code>

Flags you will use every day

-o name

name the output file

-Wall -Wextra

turn on useful warnings

-g

keep debug info for `gdb`

-O0 ... -O3, -Os

optimization level

-I dir

extra folder for headers
(preprocessor)

-L dir

extra folder for libraries (linker)

-lm

link `libm`, the math library

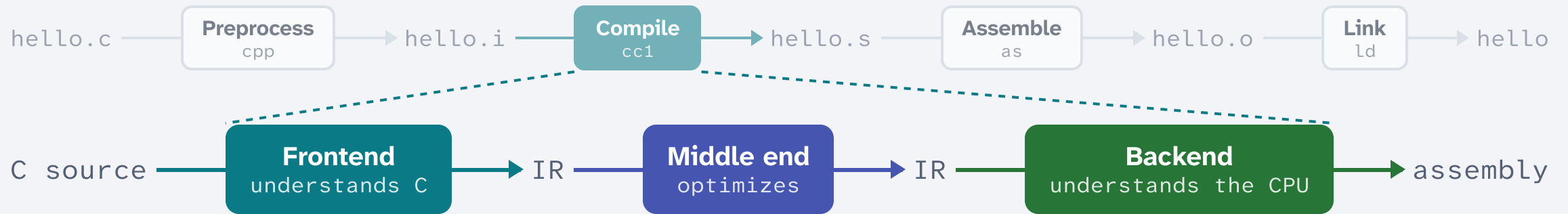
-std=c17

choose the C standard

Part 2

Inside the compiler

Zooming into the compile stage: how a compiler turns C into assembly through frontends, intermediate representations and backends.



Why split a compiler into three parts?

A shared middle language lets every source language reuse every CPU backend and every optimization.



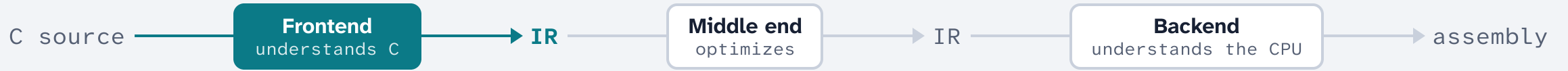
4 frontends + 3 backends = 7 pieces

Frontend knows the language: its syntax, types and rules. It reports your errors.

Middle end knows neither the language nor the CPU. It only sees IR and makes it faster or smaller.

Backend knows one CPU: its instructions, registers and calling conventions.

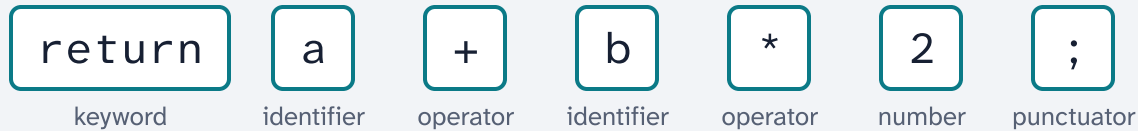
Without a shared IR, 4 languages on 3 CPUs would need **12** separate compilers. With it, **7** pieces, and a new CPU means writing one backend.



The frontend: from characters to a checked tree

1. Lexing splits text into tokens

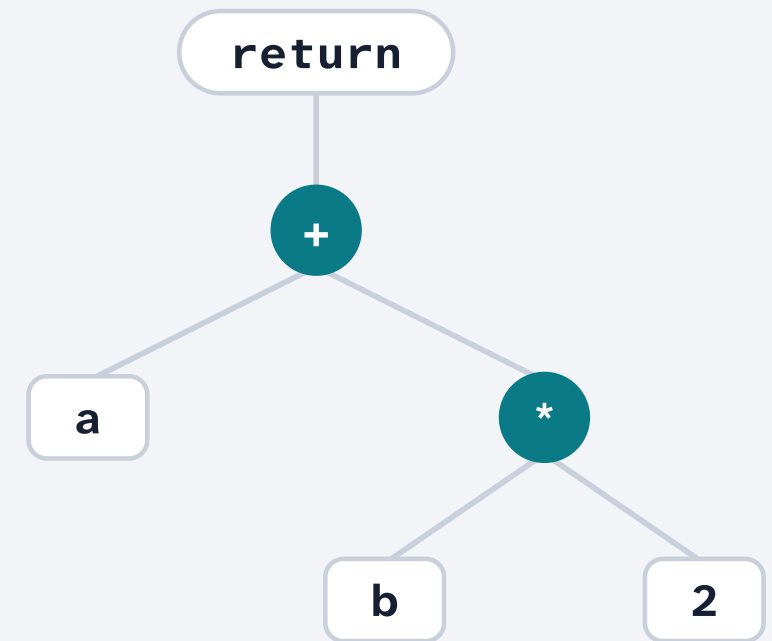
```
return a + b * 2;
```



3. Semantic analysis checks meaning

- | Are a and b declared? What are their types?
- | `int + int` gives `int`, which matches the function's return type.
- | Errors found here: undeclared identifier, incompatible types, wrong number of arguments.

2. Parsing builds a syntax tree (AST)



The tree shape records precedence: `*` sits deeper, so it happens first. No parentheses needed.

C compilers you will meet

Same language, different insides. Some compilers skip the IR completely.

Compiler	How it works inside	Where you see it
GCC	Its own IRs (GIMPLE, RTL) and its own backends	Default on most Linux systems
Clang	LLVM IR and the LLVM backends	Default on macOS, FreeBSD and Android
MSVC	Microsoft's own design, not public	Visual Studio on Windows
TCC	No IR: writes machine code while it parses	Compiles very fast, optimizes very little
chibicc	No IR: walks the syntax tree and prints assembly	A small compiler written for teaching

GCC and Clang take almost the same flags. `-E -S -c -O2 -Wall` work on both, so everything from Part 1 applies to Clang too.

No IR means little optimization. TCC and chibicc are fast and simple precisely because they skip the middle end.



What is an intermediate representation?

The compiler's own internal language: simpler than C, closer to the machine, but not tied to one CPU.

C nested expression

```
x = (a + b) * (c - d);
```

Three-address code one operation per line

```
t1 = a + b
t2 = c - d
x  = t1 * t2
```

"Three-address" because each line names at most three things: two inputs and one result.

Small steps

At most one operator per statement. Complex expressions are broken up.

Everything is named

Temporary results like `t1` get explicit names the optimizer can reason about.

Control flow is explicit

`for`, `while` and `if` become basic blocks joined by jumps.

No C-specific baggage

No preprocessor, no syntax sugar. A C++ or Fortran frontend can produce the same IR.

Inside GCC: three IRs, step by step

GCC lowers your program in stages, each one a little closer to the machine.

GENERIC

A tree for each function, much like the syntax tree.



GIMPLE

Three-address code. Most optimizations happen here.



RTL

Close to real machine instructions. Used by the backend.

f.c

```
int f(int a, int b) {  
    return a + b * 2;  
}
```

GIMPLE, as GCC prints it

```
int f (int a, int b)  
{  
    int D.2743;  
  
    _1 = b * 2;  
    D.2743 = a + _1;  
    return D.2743;  
}
```

It is the three-address code from the last slide: `b * 2` gets its own temporary, `_1`, before the addition.

D.2743 is another temporary GCC invented to hold the return value.

```
$ gcc -fdump-tree-gimple -c f.c  
$ cat f.c.*.gimple
```

The same function in LLVM IR

Clang turns C into LLVM IR. Compare it without and with optimization.

No optimization

clang -S -emit-llvm -O0 f.c

With -O1

same command

```
define i32 @f(i32 noundef %a, i32 noundef %b) {
entry:
  %a.addr = alloca i32, align 4
  %b.addr = alloca i32, align 4
  store i32 %a, ptr %a.addr, align 4
  store i32 %b, ptr %b.addr, align 4
  %0 = load i32, ptr %a.addr, align 4
  %1 = load i32, ptr %b.addr, align 4
  %mul = mul nsw i32 %1, 2
  %add = add nsw i32 %0, %mul
  ret i32 %add
}
```

```
define i32 @f(i32 noundef %a, i32 noundef %b) {
entry:
  %mul = shl nsw i32 %b, 1
  %add = add nsw i32 %mul, %a
  ret i32 %add
}
```

Without optimization, each variable lives on the stack and is loaded before use. With **-O1** that disappears, and `* 2` becomes a shift left by one, `shl`.

i32

a 32-bit integer

%a @f

a local value, a function

alloca store load

make, write and read a stack slot

nsw

assume signed math never overflows

SSA: each variable is assigned only once

Modern IRs give every assignment its own name. That makes the data flow easy to follow.

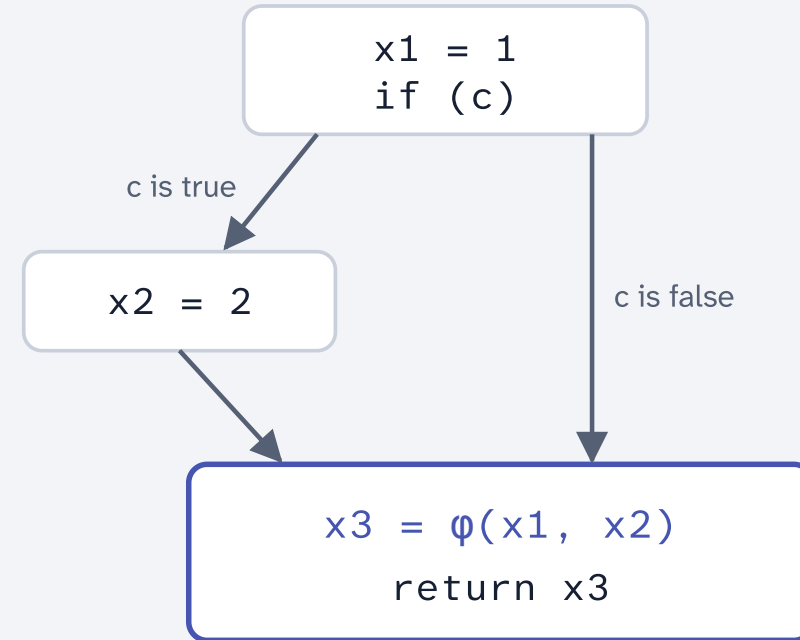
pick.c

```
int pick(int c) {  
    int x = 1;  
    if (c)  
        x = 2;  
    return x;  
}
```

x is assigned twice. Which value reaches return depends on the path taken.

In SSA form the two assignments become x1 and x2. Where the paths meet, a **φ (phi) function** picks the one that matches the path.

Why bother? With one assignment per name, the compiler always knows where a value came from. Both GCC and LLVM work in SSA form.



ϕ reads as: "x1 if we came from the top, x2 if we came through the middle box".

The middle end rewrites IR into better IR

Each pass is a small, safe transformation. Dozens run one after another.

area.c

```
int area(void) {
    int w = 3, h = 4;
    return w * h;
}
```

after -O1

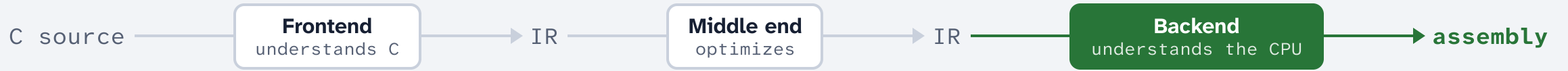
.optimized

```
int area ()
{
    <bb 2>:
    return 12;
}
```

Constant propagation and folding: the multiplication happens at compile time. gcc -O1 -fdump-tree-optimized -c area.c

Level	Use it for
-O0	Default. No optimization, easiest to debug
-Og	Light optimization that keeps debugging usable
-O2	Typical release builds
-O3	More aggressive: heavier inlining and loop work
-Os	Smaller code rather than faster code

- Constant folding and propagation**
Compute known values at compile time.
- Dead code elimination**
Delete code that is never used or never reached.
- Common subexpression elimination**
Compute a * b once if it appears twice.
- Inlining**
Paste a small function's body into its caller.
- Strength reduction**
Use cheaper operations: x * 2 becomes x<<1.
- Loop optimizations**
Move unchanging work out of loops; handle several array elements at once.



The backend: one IR, many CPUs

The same IR for `a + b * 2` becomes different instructions on each CPU.

x86-64

`gcc -O2`

```
f:
    leal    (%rdi,%rsi,2), %eax
    ret
```

Lea was made for computing addresses, but the backend uses it to do the whole sum in one instruction.

AArch64

`clang -O2`

```
f:
    add     w0, w0, w1, lsl #1
    ret
```

ARM's add can shift its second operand first: `w0 = w0 + (w1 << 1)`.

RISC-V 64

`clang -O2`

```
f:
    slli    a1, a1, 1
    addw    a0, a0, a1
    ret
```

RISC-V keeps instructions simple, so it needs two: shift, then add.

What a backend does

Picks instructions

Chooses the cheapest real instructions for each IR operation.

Assigns registers

Fits the IR's many temporaries into the CPU's few registers.

Follows the calling convention

Arguments arrive and results leave where the platform expects.

GCC and Clang, side by side

Same four stages from the outside, different machinery inside.

GCC



Clang



No separate assembler. Clang writes the object file itself. You only see a .s file if you ask with -S.

LLVM is reusable. The Rust, Swift and Zig compilers all use LLVM for optimization and code generation.

Easy to switch. Both use the system linker and the same basic flags.

Try it yourself

Everything runs on a Linux machine with GCC installed.

- 1 Run `gcc -v hello.c` and find the lines that start `cc1`, `as` and `collect2`.
- 2 Produce `hello.i`. How many lines does it have? Find `printf` in it with `grep -n`.
- 3 Write the broken `SQUARE` macro, look at it with `gcc -E -P`, then fix it.
- 4 Compare `nm hello.o` with `nm hello`. Which symbols appeared? Which still say `U`?
- 5 Leave `add.o` out of the link command on purpose. Read the error and name the stage.
- 6 Build `hello` with and without `-static`. Compare the sizes and the `ldd` output.
- 7 Compare `gcc -O0 -S f.c` with `gcc -O2 -S f.c`. What disappeared?
- 8 Do the same for `area.c` from the optimization slide. Where did the multiplication go?
- 9 On godbolt.org, compile `f.c` for x86-64, ARM64 and RISC-V. Which needs the most instructions?
- 10 Bonus: repeat exercise 1 with `clang -v`. Which program is missing?

Summary

Preprocess	. c to . i. Text substitution: includes, macros, conditionals. Knows nothing about C.
Compile	. i to . s. Frontend checks the C, middle end optimizes IR, backend writes assembly for one CPU.
Assemble	. s to . o. Machine code with holes, plus symbols and relocations that describe them.
Link	. o files + startup code + libraries into one executable; statically or dynamically.
IR and SSA	A small, explicit language between C and the CPU. One IR lets many languages share many backends.
Debugging	Read the error to find the stage: missing header, <code>file:line:col</code> , or <code>undefined reference</code> .

Going further: godbolt.org for live experiments, the GCC Internals manual for GIMPLE and RTL, and the LLVM Language Reference for LLVM IR.